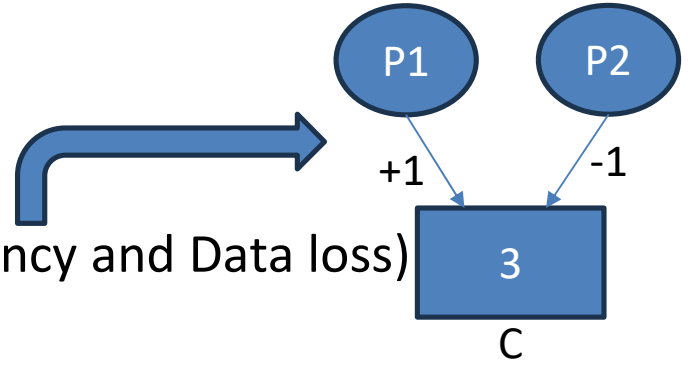
Process synchronization

- What is inter process communication (IPC)? : Can be done through files or by function through global variables

1. # Problems due to lack of synchronization→

   - Inconsistency
   - Data loss
   -Deadlock



2. # Type of synchronization→

   -Competitive → Prob: (Inconsistency and Data loss)

   - co-operative → Prob: (Deadlock)

Note: An application in an IPC environment may involve either or both type of synchronization.

- ❖ Two or more processes are said to be in **competition** iff they compete for the accessibility of shared resource.
- ❖ Here Processes 1 and 2 tries to update the value of a shared variable 'C' at the same time.
- ❑ Two or more processes are said to be in **cooperation** iff they get affected by each other i.e. the execution of one process affect the execution of other process. **Exp: Producer-Consumer Problem.**
- ❑ **Lack of cooperative synchronization among processes may lead to the problem of DEADLOCK.**

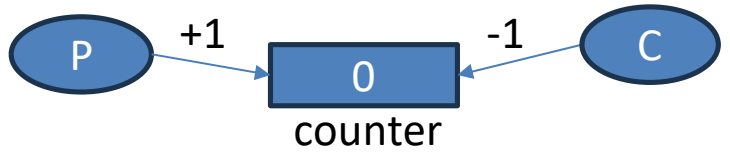
- **Illustration of the problem (Producer and Consumer)**:

Suppose that we wanted to provide a solution to the producer-consumer problem that fills all the buffers.

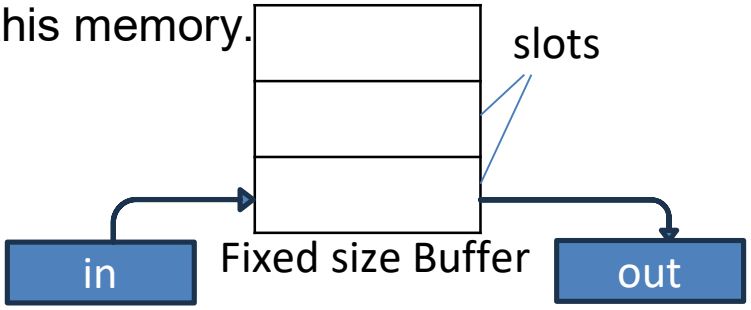
- We can do so by having an integer counter that keeps track of the number of full buffers. Initially, counter is set to 0.
- It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.

- Processes can execute concurrently.
- May be interrupted at any time, partially completing execution.
- Concurrent access to shared data may result in data inconsistency.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes

Producer and Consumer Problem:



Both processes access this memory.



In operating System Producer is a process which is able to produce data/item.

```
#define N 100          /Global Declarations
Int Buffer [N];          /*shared by P&C
Int count =0;            /*shared by P&C

Void producer (void)
 {
   int temp, in=0;
   While (True)
    {
       /* produce an item in temp */
       produce item (item p);
       while (count = = N);
              buffer[in] = temp;
               in = (in + 1)%N
              count= count+1;
   }
}
```

Co-operation

In operating System consumer is a process which is able to consume data/item.

```
Void consumer (void)
 {
   int temp, out=0;
   While (True)
    {
       while (count == 0) ;
              item c = Buffer [out];
               out = (out + 1)%N
              count= count-1;
              Process item [item c]
   }
}
```

Competition

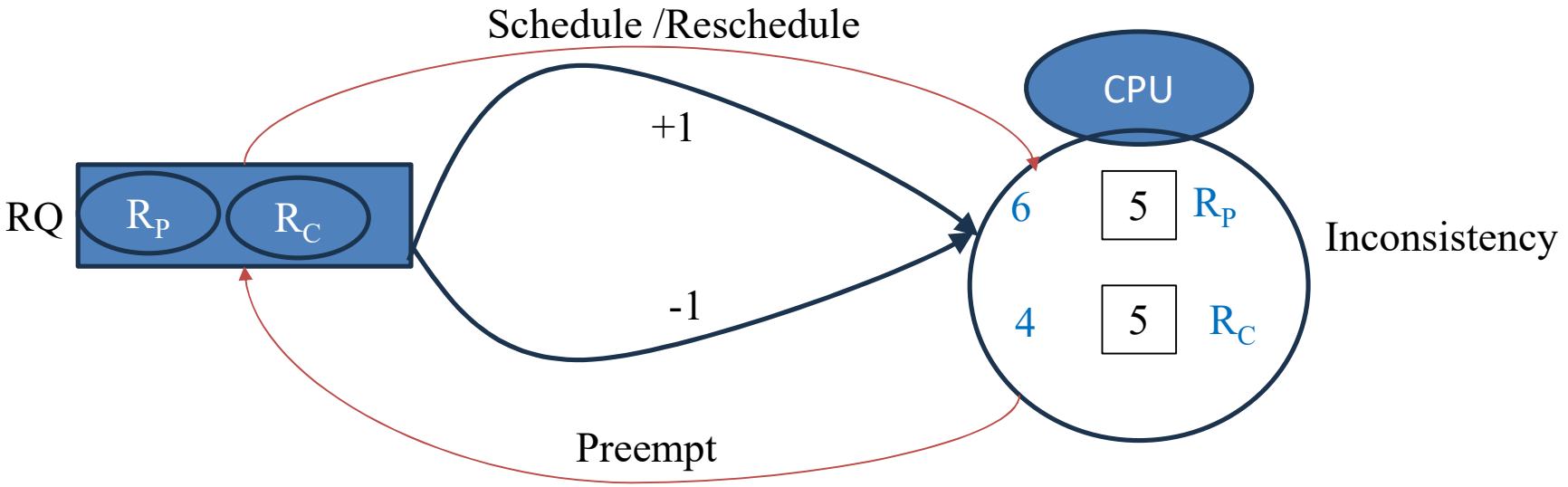
IN indicates the location where the item suppose to be placed in NEXT. “IN” is locating already next position. So, this sequence is right to place items.

SETUP:

I. Load R_P, M[count]
II. Inc, R_P
III. SM[Count], R_P

I. Load R_C, M[count]
II. Dcr, R_C
III. SM[Count], R_C

$T_i \rightarrow R_P$: I, II, Preempt
$T_j \rightarrow R_C$: I, II, III
$T_k \rightarrow R_P$: III

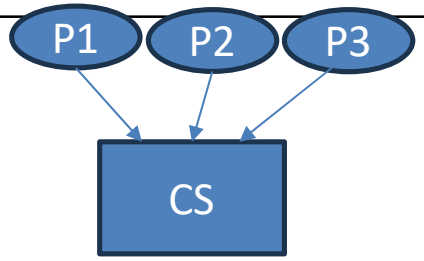


Producer Consumer Problem

- Producer-Consumer problem is a **classical synchronization problem** in the operating system. With the presence of more than one process and limited resources in the system the synchronization problem arises.
- If one resource is shared between more than one process at the same time then it can lead to data inconsistency.
- In the producer-consumer problem, the producer produces an item and the consumer consumes the item produced by the producer.
- Both Producer and Consumer share a common memory buffer. This buffer is a space of a certain size in the memory of the system which is used for storage. The producer produces the data into the buffer and the consumer consumes the data from the buffer.
- Accessing memory buffer should not be allowed to producer and consumer at the same time.
- first, understand the few terms used in the code:
- "in" used in a producer code represent the next empty buffer.
- "out" used in consumer code represent first filled buffer.
- count keeps the count number of elements in the buffer.

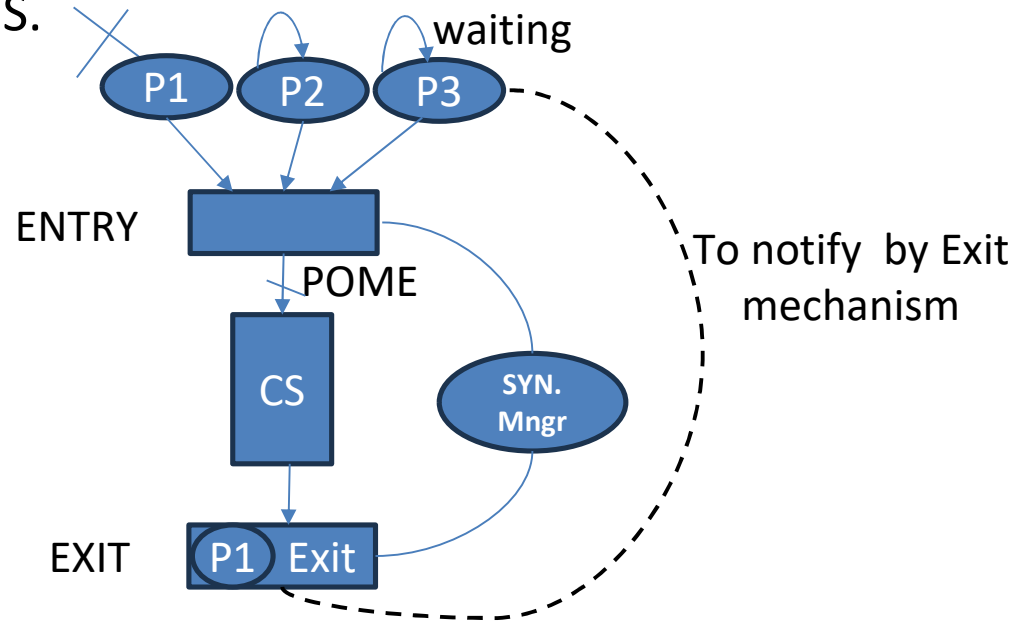
3. # Causes responsible for synchronization problems→ (Necessary conditions)

critical section (CS):	part of program where shared resources are accessed. (NonCS: which does not consist shared resources) A program in IPC may involve or may have a series of CS and NonCS
Race condition:	Process must be racing either competitively or cooperatively to access CS, exp: P&C problem, tries to update the count at the same time.
Preemption	

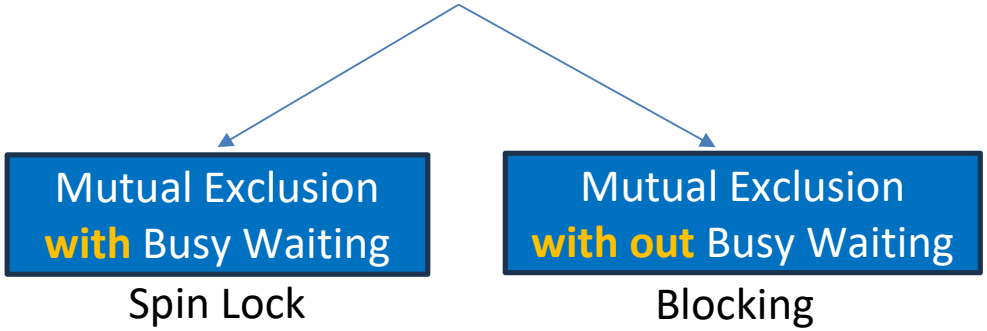


Principal of Mutual Exclusion (POME):

- If a process is present in C.S., other process should not be allowed to enter the C.S. (should be waiting outside the C.S.)
- POME is ensured by syn. Mechanism or synchronization Tool, placed between processes and C.S.



Classification of synchronization Mechanism:



Software solution

- Lock variable
- Peterson
- Strict alteration

Hardware solution

- Test and Set Lock (TSL)
- Swap

OS based solution

- Semaphore

Solution	Processes	Practical
Lock variable	N	
Strict alternation	2	
Peterson's	2	
Mutex/Semaphore	N	More

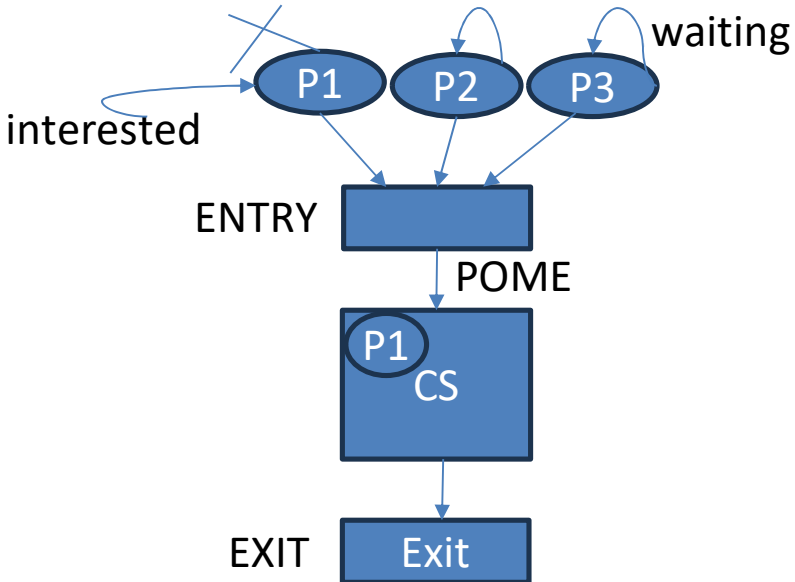
Requirement of Synchronization Mechanism for solving Critical Section Problem

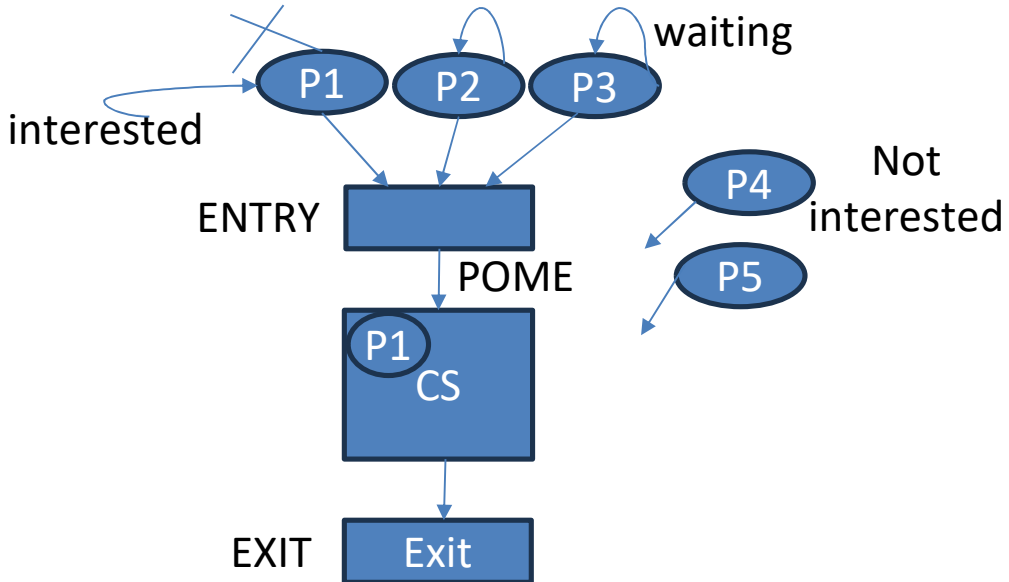
1. Mutual Exclusion: must always be **guaranteed**. //(else inconsistency/data loss)

- "**All** processes are interested"

2. Progress-

- "**Some** processes are interested"
- **Def1**: No process running outside CS should **block** the other interested processes from entering CS.
- **Def.2**: Processes running outside CS (Non-CS) **should not participate** in decision making of allowing a process to enter CS, among those who are interested.

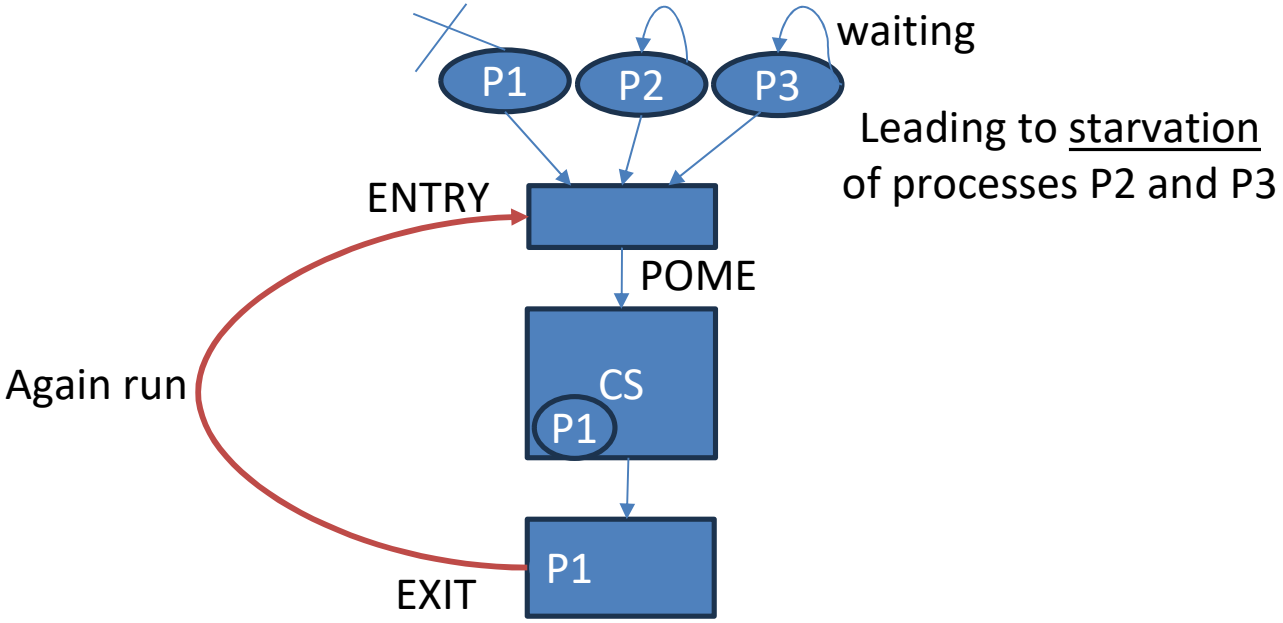




Cont.

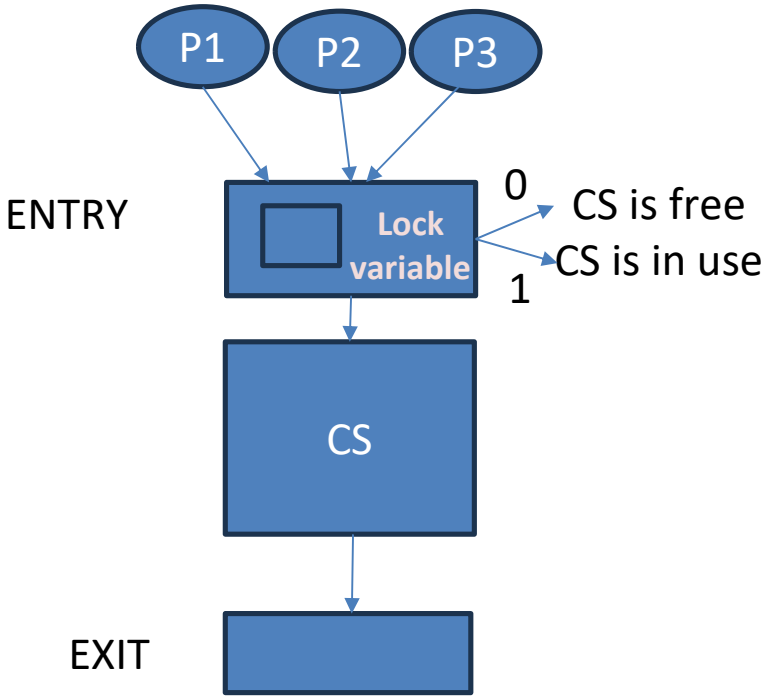
3. Bounded Waiting-

- "All processes are interested"
- **Def1**: No process has to **wait forever** to access its **CS**.
- **Def.2**: There **should be a bound** on number of times a process is allowed to enter CS before the other waiting process's request is granted.



Lock Variable

- **ME with Busy waiting &S/W solution**
- **Multi process solution, >2 processes**
- Busy waiting solutions are always implemented using loop mechanism for repeatedly checking.



- Int Lock =0;
- Void process (int i)
- {
- While (1)
- {
- a) Non-CS ();
- b) While (Lock!=0);
- c) Lock=1
- d) <C.S.>
- e) Lock =0;
- }
- }

Busy waiting

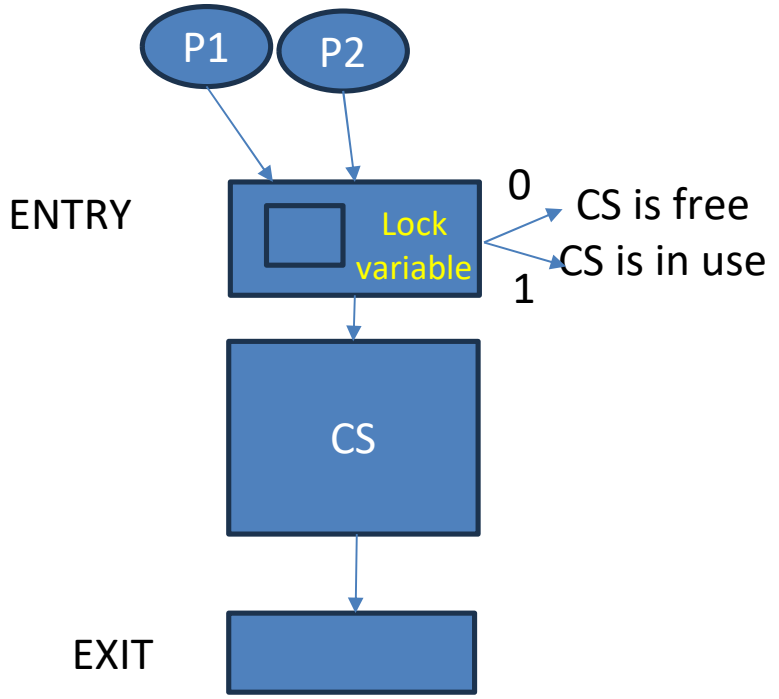
ENTRY

EXIT

Cont.

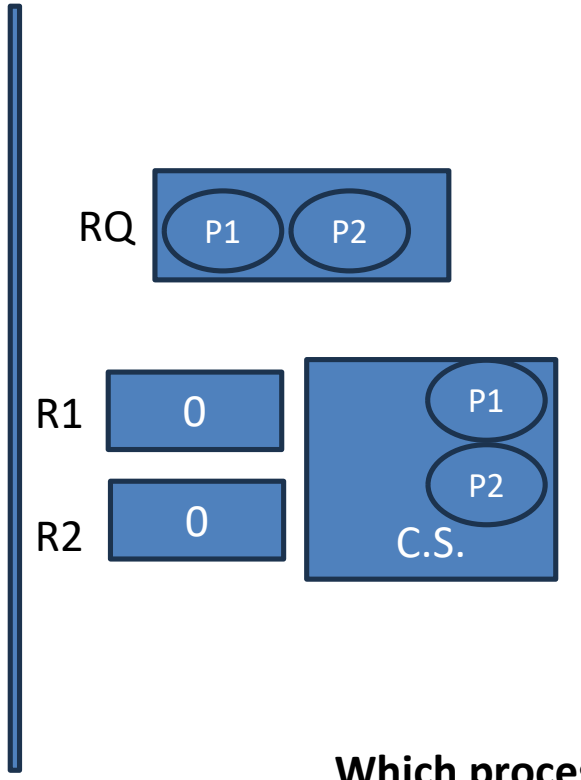
- ME guaranteed →no,

  only if No preemption
- Progress guaranteed →yes
- BW guaranteed →no (in either case of Pre/Non-Preemption)



SETUP:

- I. load Ri, M [Lock]
- II. Compare Ri, #0;
- III. Jump IF not Zero, Step I
- IV. Store M[Lock], #1;
- V. <CS>
- VI. Store M[Lock], #0

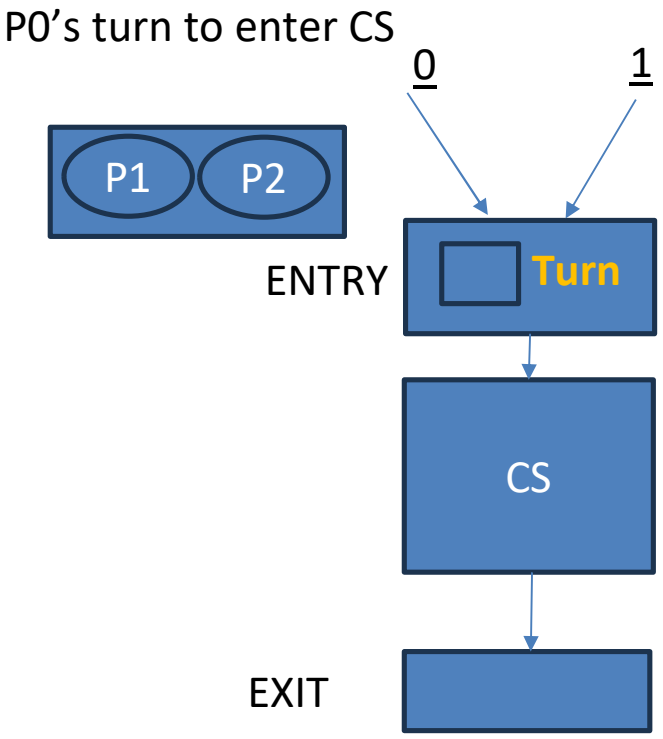


- ti : P1 : I, II, III
- tj: P2 : I, II, III, IV, V
- tk: P1 : IV, V

Which process will enter to CS, first ?

Strict Alteration

- **ME with Busy waiting &S/W solution**
- **2 process solution (P0, P1; Pi, Pj)**
- Strictly on alternate basis, processes takes turn to enter C.S.



- Int turn = rand (0,1);
- Void process (int i)
- {
- While (1)
- {
- a) Non-CS ();

ENTRY
- b) While (turn!=i);
- c) <C.S.>

EXIT
- d) turn =j;
- }
- }

- ME guaranteed →yes
- Progress guaranteed →no
- BW guaranteed →yes

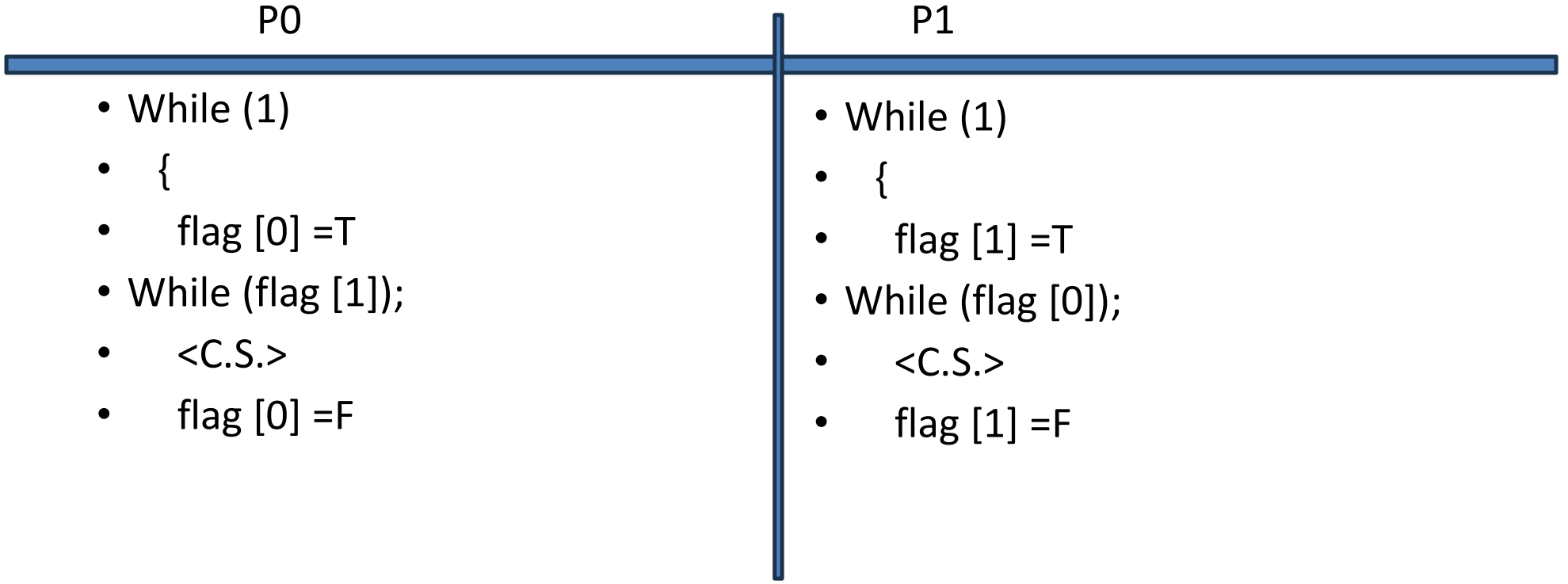
➢ First enter CS then after coming out change the value of ID unlike LOCK variable.

Observations

- It does **not guarantee Progress** due to the fact that a process which is not interested in CS having its ID set to turn **prevent the other interested process from accessing CS**.

- In general, an interested process must be allowed to enter CS **anytime** and **any number of times as long as CS is free** and other processes are not interested.

Another solution



- ME-Yes
- Progress-No // both interested at same time, else Yes
- BW-No // If both set their flags at the same time

➢ It will lead to deadlock if both becomes interested at same time.

Peterson's Solution

- ME with Busy waiting &S/W solution
- 2 process solution (P0, P1; Pi, Pj)
- Employs advantages of both:

(Lock Variable and Strict Alteration).

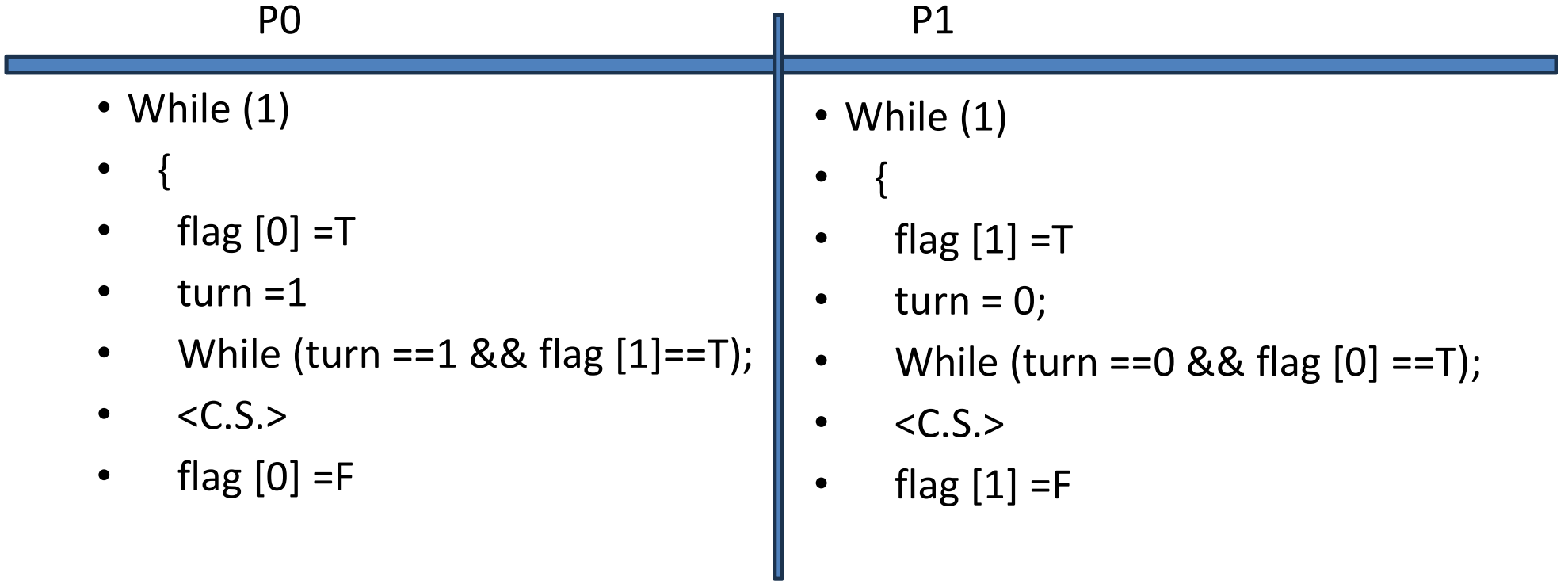
```
# define N 2          //two processes only
# define TRUE 1  //define Boolean values
# define FALSE 0
Int interested [N] = {FALSE};  //shared variable, initially No process want to enter CS
Int turn;                          //shared variable
Void process (int i)        // i can be 0 or 1
{
  While (1)
   {
     a)  Non-CS ();   // Both processes can execute this simultaneously.
     b)  interested [i] =TRUE;  // Process i announces that it wants to enter into the critical section.
     c) turn =i;
     d) While (interested [J] == TRUE && turn ==i);// Bust wait condition
     e)   <C.S.>
     f) interested [i] =FALSE;
    }
}
```

// interested[0] = TRUE → Process 0 wants to enter CS.
// interested[1] = FALSE → Process 1 does not want to enter CS.

- ti : P0 : a, b
- tj: P1 : a,b,c,d
- tk: P0 : c,d //i=0, j=1
- tl: P1 : d, e
- //i=1, j=0

- ti : P0 : a, b, c, d, e //considering P1 is not interested
- //P1 is interested
- tj: P1 : a, b, c, d Busy wait, ensuring ME
- tk: P0 : e, f, a, b, c, d

Peterson's Solution



P0

- While (1)
- {
- flag [0] =T
- turn =1
- While (turn ==1 && flag [1]==T);
- <C.S.>
- flag [0] =F

P1

- While (1)
- {
- flag [1] =T
- turn = 0;
- While (turn ==0 && flag [0] ==T);
- <C.S.>
- flag [1] =F

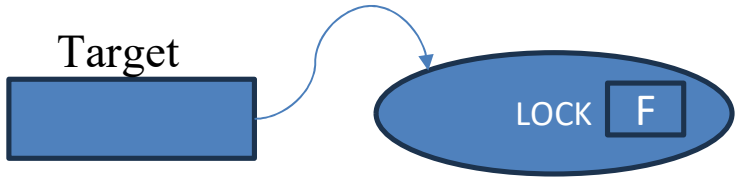
- ME guaranteed →yes
- Progress guaranteed →yes // If P1 is not interested then it is not able to block P0 entering into CS
- BW guaranteed →yes

Test & Set Instructions:-

- In a **Uniprocessor environment**→ no concurrent execution,
- ME can be obtained by preventing process, that is running in Critical Section (CS), from being interrupted. By→ disabling the interrupt before process enters into CS.
- Enable the interrupts only after the process exited from CS.
- When a process is executing in its CS, it can not be interrupted and thus ME is achieved.
- However, **in multiprocessor environment**, disabling all interrupts by sending interrupt signal to all processor is time consuming. Therefore, to achieve ME, Test and Set instructions are used.
- Test and Set instructions are simple H/W instructions that solve the CS problem by providing ME in an easy and efficient way, in Multiprocessor environment.

➢ The synchronization mechanism is implemented **directly in the CPU hardware**, not in software. **Test & Set** is an **atomic machine instruction** provided by the processor.

TSL:



- ME with Busy waiting &H/W solution
- Multi-process solution
- Special h/w supported instruction
- **Extended Lock Variable.**

```
Bool TSL (Bool *Target)
{
    Bool r.v.;
    a)  r.v. =*target;
    b)  *Target =TRUE;
    c) return (r.v.);
}
```

- TSL returns current value of Lock (pointed by Target) and Sets the value of Lock always TRUE

```
Bool Lock =FALSE;   //GLOBAL variable
Void process (int i)
{
   While (True)
    {
      a) Non-CS ();
      b) While (TSL (&Lock)==True);
      c) <CS>
      d) Lock =FALSE;
    }
}
```

- Ensures ME
- Ensures Progress
- Not Bounded waiting
- Deadlock free but not starvation free

//process tries to acquire the lock using TSL

- t0 : P1 : a, b :TSL=F
- t5: P2 : a, b :TSL=T

- Guarantee ME

TSL:

- I.
- II.
- III.
- IV.
- V.

- I. load Ri, M [Lock]
- II. Store M[Lock], #1;
- III. Compare Ri, #0;
- IV. Jump IF not Zero, Step I
- V. <CS>
- VI. Store M[Lock], #0

- **Atomic**, i.e. no preemption between these two statements

In LOCK variable

LOCK:

- ME guaranteed →no,
  only if No preemption
- Progress guaranteed →yes
- BW guaranteed →no (in either case of Pre/Non-Preemption)

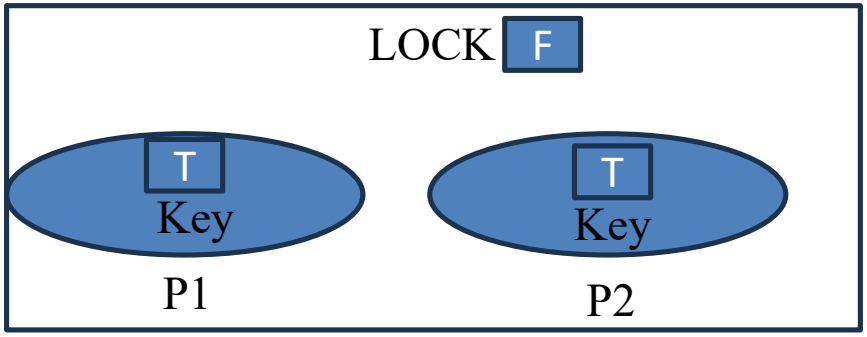
TSL:

- Ensures ME
- Ensures Progress
- Not Bounded waiting

hardware-supported **Swap** instruction

Swap Instruction:

```
Void swap (Bool *a, Bool*b)
{
  Bool t;
  t =*a;          temp = key
*a=*b             key = Lock
*b=t;             Lock = temp
}
```



- Ensures ME
- Ensures Progress
- Not Bounded waiting

Swap is:
- An **atomic machine instruction**
- Executed directly by the CPU
- Cannot be interrupted in the middle
- This atomicity is provided by **hardware support**, not by the programmer.

```
Bool Lock =FALSE;    // Shared variable
void process (int i)
{
   Bool key;   //local variable
   While (1)
     {
        a) Non-CS ();
        b) key=True;     //interest
        c) while (key =TRUE)
            {
             Swap (&key, &Lock);     //a special CPU instruction
            }
       d) <CS>
       e) Lock =FALSE;
     }
}
```

Case 1: Lock is FREE (Lock = FALSE)
- key = TRUE
- Lock = FALSE

After swap:
- key = FALSE
- Lock = TRUE

Now key == FALSE, so loop exits.
Process enters Critical Section.

Case 2: Lock is BUSY (Lock = TRUE)
- key = TRUE
- Lock = TRUE

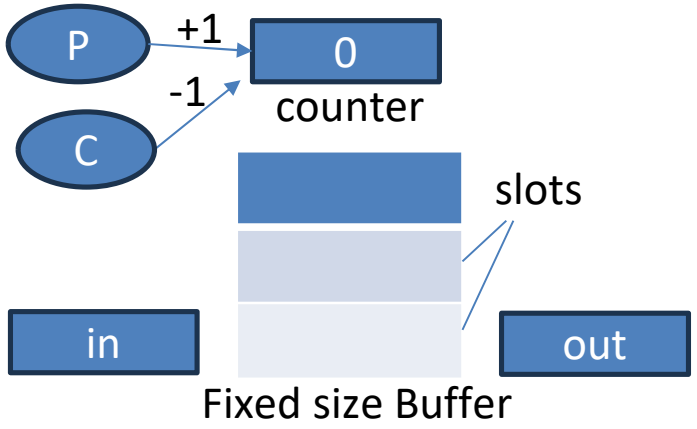
After swap:
- key = TRUE
- Lock = TRUE

Nothing changes → loop continues.

Mutual Exclusion with Blocking/Without Busy wait

1. **Sleep ()& Wake up();**

-----Are OS primitives
-----ME with Blocking
-----Multi-process Solution




```
Void producer (void)
 {
   int itemp, in=0;
   While (1)
    {
        /* produce an item in temp */
        a) Produce item (itemp);
         b) if(count== N) sleep();
        c) Buffer [in]= itemp;
        d) in = (in + 1) %N
         e) count= count+1;
         f) if (count==1) wakeup(consumer);
    }
}
```

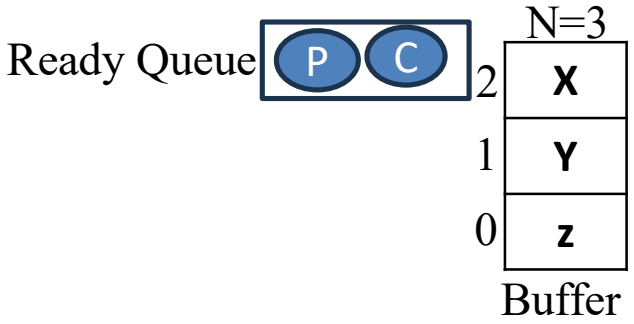
Co-operation

```
{
a) if(count == 0)sleep ();
b) itemp = Buffer [out]
c) out = (out + 1)%N;
d) count= count-1;
e) If (count==N-1) wakeup(producer);
f) process item (item c);
}
```

Competition

Sleep Queue

- One process wakeups the other
- This code also leading to deadlock



count 0,1,2, 3